In [1]:
```
# Initialize Otter
import otter
grader = otter.Notebook("hw9.ipynb")
```

CPSC 330 - Applied Machine Learning

Homework 9: Communication

Due date: See the Calendar

Instructions

rubric={points}

You will earn points for following these instructions and successfully submitting your work on Gradescope.

Group work instructions

You may work with a partner on this homework and submit your assignment as a group. Below are some instructions on working as a group.

- The maximum group size is 2.
- Use group work as an opportunity to collaborate and learn new things from each other.
- Be respectful to each other and make sure you understand all the concepts in the assignment well.
- It's your responsibility to make sure that the assignment is submitted by one of the group members before the deadline.
- You can find the instructions on how to do group submission on Gradescope here.
- If you would like to use late tokens for the homework, all group members must have the necessary late tokens available. Please note that the late tokens will be counted for all members of the group.

General submission instructions

- Please **read carefully Use of Generative AI policy** before starting the homework assignment.
- **Run all cells before submitting:** Go to `Kernel -> Restart Kernel and Clear All Outputs`, then select `Run -> Run All Cells`. This ensures your notebook runs cleanly from start to finish without errors.
- **Submit your files on Gradescope.**
  - Upload only your `.ipynb` file **with outputs displayed** and any required output files.
  - Do **not** submit other files from your repository.
  - If you need help, see the Gradescope Student Guide.
- **Check that outputs render properly.**
  - Make sure all plots and outputs appear in your submission.
  - If your `.ipynb` file is too large and doesn't render on Gradescope, also upload a PDF or HTML version so the TAs can view your work.
- **Keep execution order clean.**
  - Execution numbers must start at "1" and increase in order.
  - Notebooks without visible outputs may not be graded.
  - Out-of-order or missing execution numbers may result in mark deductions.
- **Follow course submission guidelines:** Review the CPSC 330 homework instructions for detailed guidance on completing and submitting assignments.

Points: 2

Exercise 1: Survival analysis

rubric={points}

The following questions pertain to the lecture on survival analysis. We'll consider the use case of customer churn analysis.

1. What is the problem with simply labeling customers are "churned" or "not churned" and using standard supervised learning techniques?
2. Consider customer A who just joined last week vs. customer B who has been with the service for a year. Who do you expect will leave the service first: probably customer A, probably customer B, or we don't have enough information to answer? Briefly explain your answer.
3. If a customer's survival function is almost flat during a certain period, how do we interpret that?

Solution_1

Points: 6

1. Simply labelling customers as "churned" or "not churned" ignores right-censoring and leads to biased estimates. This approach treats customers who have not yet churned as if they will never churn, but removing them will also discard useful information. As a result, our dataset becomes a biased sample of those who churned within the time window of the data collection.
2. With just the information given, there is not enough information to answer. Churn risk depends on the behaviour of the customers towards that specific service. It might be higher in the first week while people decide whether they like the product, or it might increase after a long period if the customers no longer benefit from it.
3. If a customer's survival function is flat, then the likelihood of churn during that period is very low. This means that their probability of staying the same from week to week is stable, and it is unlikely for there to be changes in churn during that time.

Exercise 2: Communication

2.1 Blog post

rubric={points}

Write up your analysis from hw5 or any other assignment or your side project on machine learning in a "blog post" or report format. It's fine if you just write it here in this notebook. Alternatively, you can publish your blog post publicly and include a link here. (See exercise 4.3.) The target audience for your blog post is someone like yourself right before you took this course. They don't necessarily have ML knowledge, but they have a solid foundation in technical matters. The post should focus on explaining **your results and what you did** in a way that's understandable to such a person, **not** a lesson trying to teach someone about machine learning. Again: focus on the results and why they are interesting; avoid pedagogical content.

Your post must include the following elements (not necessarily in this order):

- Description of the problem/decision.
- Description of the dataset (the raw data and/or some EDA).
- Description of the model.
- Description your results, both quantitatively and qualitatively. Make sure to refer to the original problem/decision.
- A section on caveats, describing at least 3 reasons why your results might be incorrect, misleading, overconfident, or otherwise problematic. Make reference to your specific dataset, model, approach, etc. To check that your reasons are specific enough, make sure they would not make sense, if left unchanged, to most students' submissions; for example, do not just say "overfitting" without explaining why you might be worried about overfitting in your specific case.
- At least 3 visualizations. These visualizations must be embedded/interwoven into the text, not pasted at the end. The text must refer directly to each visualization. For example "as shown below" or "the figure demonstrates" or "take a look at Figure 1", etc. It

is **not** sufficient to put a visualization in without referring to it directly.

A reasonable length for your entire post would be **800 words**. The maximum allowed is **1000 words**.

Example blog posts

Here are a few optional examples if you'd like inspiration. You don't need to read them in full; instead, skim one and pay attention to:

- how they introduce the question/problem,
- how they describe the data and show a few key plots,
- how they briefly describe the model without turning it into a tutorial, and
- how they discuss results and limitations in plain language.

Examples

- Will your Kickstarter Project be successful? | A simple analysis to help you predict better!!! (GoodAudience)
- My Little Markov Model - Now Tweeting New Taylor Swift Lyrics (UBC MDS)
- What's for dinner? Predicting customer order probabilities (UBC MDS)

Additional inspiration

These are interviews with Kaggle competition winners. They aren't blog posts, but they offer insight into how people articulate data, modeling decisions, and results:

- Instacart Market Basket Analysis
- Winner Interview with Shivam Bansal | Data Science for Good Challenge: City of Los Angeles

A note on plagiarism

You may **NOT** include text or visualizations that were not written/created by you. If you are in any doubt as to what constitutes plagiarism, please just ask. For more information see the UBC Academic Misconduct policies. Please don't copy this from somewhere or ask Generative AI to write it for you 🙏. Please carefully read the Use of Generative AI policy before writing your blogpost.

Solution_2_1

Points: 26

What is Airbnb?

Airbnb is an online platform that enables property owners to rent out their accommodations to guests for both short-term and long-term stays. It is widely used by travellers around the world who are looking for places to stay during their trips. From private bedrooms, apartments, vacation homes, boutique hotels, cottages and more, Airbnb allows guests to search for their right accommodation based on their desired location, with the features they want and within their price range easily.

For hosts, especially homeowners, the platform makes it easy to list their properties and earn money from spaces that might otherwise go unused. In addition to photos, descriptions and host information such as Superhost status, Airbnb also allows both guests and hosts to write reviews and rate each other too. This creates a sense of mutual credibility and acts as a form of verification. Reviews help future guests understand what the property is actually like beyond what the host has listed, giving them real insight from previous visitors.

Since more popular listings typically accumulate a higher number of reviews, this creates an opportunity to develop a predictive model that estimates the potential popularity of new listings before they are posted. Hence, this could potentially help hosts create more appealing listings from the start.

Problem Definition and Data Understanding

For my HW5 project, I used the Kaggle dataset of New York City Airbnb listings from 2019. This data includes information about each listing, the host, geographical attributes, and various other metrics related to availability and performance. The features included in the dataset, along with their descriptions, are as follows:

- **id**: A unique identifier for the listing.
- **name**: A descriptive name of the listing.
- **host_id**: A unique identifier for the host.
- **host_name**: The name of the host.
- **neighbourhood_group**: The broader districts within New York City (Brooklyn, Manhattan, Queens, Bronx, Staten Island)

- **neighbourhood**: The specific neighbourhood where the listing is located (for example, Williamsburg, Bedford-Stuyvesant, Harlem, Bushwick, Upper West Side).
- **latitude**: The latitude coordinate of the property.
- **longitude**: The longitude coordinate of the property.
- **room_type**: The type of space being listed, such as the entire home or apartment, a private room or a shared room.
- **price**: The price per night in dollars.
- **minimum_nights**: The minimum number of nights required for a booking.
- **number_of_reviews**: The total number of reviews the listing has received.
- **last_review**: The date of the most recent review.
- **reviews_per_month**: The average number of reviews the listing receives per month.
- **calculated_host_listings_count:** The total number of listings owned by the host.
- **availability_365**: The number of days in a year that the listing is available for booking. In this project, I will be predicting the variable `reviews_per_month` as a proxy for the popularity of the listing. A model like this could help Airbnb estimate the potential popularity of future listings they are planning to post, which may also guide hosts in creating more appealing and competitive listings.

Exploratory Data Analysis

Before building my model, I conducted an exploratory data analysis to better understand the dataset and the characteristics of each variable. From this analysis, I observed that name, host_name, neighborhood_group, neighborhood, and room_type are categorical features, while last_review is a date feature. The columns latitude, longitude, price, minimum_nights, number_of_reviews, calculated_host_listings_count, and availability_365 are numerical features. The target variable for this project is reviews_per_month, and the columns id and host_id are simply identifiers and can be safely dropped.

The values for reviews_per_month range from 0.01 to 58.50. A value of 0.01 is effectively equivalent to zero reviews, since reviews cannot be a decimal number. During this analysis, I also found that several features contained missing or null entries stored as "NaN". As a result, data imputation will be required to ensure that these missing values are handled appropriately. Entries with a missing value for reviews_per_month should also be dropped because data imputation would simply be the same as the purpose of the model that we are building.

As seen in the Figure 1 below, I also observed a strong relationship between price and reviews_per_month. Lower-priced listings generally receive more reviews per month. The distribution of price is also heavily right-skewed, with many listings clustered at lower price points and fewer listings at higher prices.

Figure 1. Price Distribution



A similar trend also appears for minimum_nights. Listings with lower minimum stay requirements tend to accumulate more reviews per month. This feature is also right-skewed, with most hosts setting relatively low

Figure 2. Minimum Nights



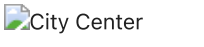
Feature Engineering

Based on preliminary research on Airbnb and how guests choose their accommodations, I found that proximity to the city center is an important factor in the decision-making process. Because of this, I decided to use the latitude and longitude feature listing's location to compute its distance from the center of New York City. Travellers often consider how close a property is to the city center, as proximity generally influences convenience and accessibility. Therefore, creating a new feature that measures the distance of each listing from the city center could add valuable predictive power to the model.

The coordinates of central Manhattan are approximately longitude -73.9794 and latitude 40.7639. Since one degree of latitude or longitude corresponds to roughly 111km, I used the Euclidean distance formula with a conversion factor of 111km per degree to calculate the distance of each listing from the city center.

Using this formula, along with the code shown in Figure 3 below, I computed the distance from each listing to the center of New York City.

Figure 3. Minimum Nights



Preprocessing and Transformations

As mentioned previously, some instances in the dataset contain missing target values. Since models cannot be trained on unkown target variables, these rows must be removed. For simplicity, I also dropped rows with missing values in the reviews_per_month

feature.

Next I applied one-hot encoding to the categorical variables so they could be converted into a numerical format that the model can interpret. For the numerical features, I used a Simple Imputer with the mean strategy to fill in any of the remaining missing values. Then, I applied a StandardScaler to normalize all numerical features, ensuring that they are on a comparable scales and weighted fairly during model training.

Finally, I dropped the features id, host_id, host_name, latitude, longitude, last_review, calculated_host_listings_count, and name. As mentioned earlier, id and host_id can be safely removed because they are simply unique identifiers and do not contribute meaningful information to the model. Similarly, host_name and name are also unique to each listing or host and do not provide useful predictive value. The latitude and longitude columns have already been used to create the engineered feature distance_to_center, so they are no longer necessary. The feature calculated_host_listings_count showed no clear relationship with reviews_per_month during exploratory analysis, so it was removed as well. Lastly, the name feature was dropped because it is unlikely to have strong predictive power and would require additional text preprocessing steps that are not essential for this project.

Model Creation

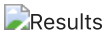
Once all the features were created and preprocessed, I began model creation and testing.

The list of models I explored is as follows:

- Dummy Regressor (baseline model)
- Linear Regression with Grid Search and Randomized Search for hyperparameter optimization
- RandomForestRegressor
- GradientBoostingRegressor
- RGBRegressor

The results are summarized below.

Figure 4. Summary of Results



Interpretation of Results

Since this is a regression problem, suitable evaluation metrics include Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and R^2. Because the goal is to predict the popularity of a listing using the target feature reviews_per_month, RMSE is the most appropriate primary metric. It is expressed in the same units as the target variable and penalizes large errors more heavily, making it useful for capturing significant deviations. A lower RMSE indicates better performance, while a higher RMSE reflects poorer predictive accuracy.

Based on the results, the best-performing model for the New York City Airbnb dataset is XGBoost, with a test RMSE of 1.124 and a test R^2 of 0.0526. Random Forest and Gradient Boosting performed similarly, with test RMSE values around 1.17.

Caveats of the model

1. reviews_per_month is an imperfect measure of popularity. Not all guests leave reviews unless they had a very positive or very negative experience. Review frequency can also vary based on guest demographics or the type of listing. As a result, two equally popular listings may have very different review rates, making this an imperfect representation for true demand.

2. Imputation and data removal may introduce bias. Missing values were imputed using the mean under the assumption that it reasonably represented the data. However, this may oversimplify the true distribution and reduce variation. Additionally, rows with missing target values had to be removed, which may have led to an uneven data distribution or subtle bias in the training set.

3. Some dropped features may have contained valuable information. For simplicity, several features were removed, but many of them might still hold predictive value when used correctly. For example, a descriptive listing title could indirectly reflect amenities, style, or uniqueness, all of which might influence guest decisions. Removing these features simplifies processing but may limit predictive performance.

Key Takeaways

Throughout this assignment, I learned how essential proper feature preprocessing is for building effective machine learning models. Without careful preprocessing, issues can arise during feature selection, scaling, and hyperparameter optimization, ultimately affecting model performance.

Exploring different models showed that while ensemble tree-based methods are more complex, they tend to perform better overall. The accuracy gains were significant enough to justify the added complexity. However, even after hyperparameter tuning and feature reduction, the improvements across models remained relatively small, suggesting that the current feature set and model choices already capture most of the available predictive signal in the dataset.

For future work, I hope to conduct more extensive hyperparameter optimization for XGBoost and incorporate interpretability tools such as SHAP and ELI5 to better understand feature contributions and improve model transparency.

2.2 Effective communication technique

rubric={points}

Describe one effective communication technique that you used in your post, or an aspect of the post that you are particularly satisfied with. (Max 3 sentences.)

Solution_2_2

Points: 4

In my post, I divided the analysis into sections labelling them with a heading to break the content into smaller, more easily interpretable parts. This structure helps users understand the information more clearly and take away the key points more easily.

(optional, not for marks) 2.3

Publish your blog post from 1.1 publicly using a tool like Quarto, or somewhere like medium.com, and paste a link here. Be sure to pick a tool in which code and code output look reasonable. This link could be a useful line on your resume!

Solution_2_3

Points: 0

Exercise 3: Your takeaway from the course

rubric={points}

Your tasks:

- Reflect on your journey through this course. Please identify and elaborate on at least three key concepts or experiences where you had an "aha" moment. How would you use the concepts learned in this course in your personal projects or how would you approach your past projects differently based on the insights gained in this course? We encourage you to dig deep and share your genuine reflections.

  > Please write thoughtful answers. We are looking forward to reading them 🙂 .

Solution_3

Points: 2

What resonated with me most throughout this course was learning about the models we regularly interact with in the real world. Topics like clustering, recommender systems, and time series analysis were especially interesting because I could clearly see their purpose and how they are applied. It was also eye-opening to understand how these models work on the back end after having used and been affected by them myself. Overall, it highlighted how useful and intentional machine learning is in real-world applications and reinforced the direct relevance of what we learned in class.

Congratulations 👏👏

That's all for the assignments! Congratulations on finishing all homework assignments!

In [2]:
```
from IPython.display import Image

Image("img/eva-congrats.png")
```

Out[2]:

